

Análisis de verbatims mediante procesamiento de lenguaje natural y clasificación multietiqueta

Franco Reinoso – Gustavo Montoya

Universidad Técnica Federico Santa María

Introducción al Aprendizaje Automático

Profesor: Alejandro Veloz

3 de julio de 2026

Resumen

El presente proyecto analiza comentarios abiertos, también denominados *verbatim*, mediante técnicas de procesamiento de lenguaje natural y clasificación multietiqueta. El trabajo se organiza en dos líneas experimentales desarrolladas de manera independiente sobre conjuntos de datos provenientes de dominios distintos. En ambos casos, el flujo considera estructuración de datos, limpieza textual, generación de variantes de preprocesamiento, extracción de características, entrenamiento, ajuste de hiperparámetros, evaluación y análisis de resultados.

La primera línea de trabajo, desarrollada sobre comentarios de una encuesta de satisfacción del sector seguros, transformó cuatro preguntas abiertas a formato largo y obtuvo 6.119 comentarios válidos. Sobre una muestra manualmente etiquetada de 1.500 comentarios se evaluaron cinco variantes de texto y dos representaciones numéricas: TF-IDF con n-gramas y Word2Vec. Las combinaciones resultantes se evaluaron con K-Nearest Neighbors (KNN) en un escenario de clasificación multietiqueta. Antes de calibrar los umbrales de decisión, el mejor resultado en prueba correspondió a TF-IDF con texto lematizado, con precisión macro de 0.9177, recall macro de 0.7377 y F1 macro de 0.8133. Posteriormente, una etapa de ajuste de umbrales por categoría seleccionó TF-IDF con stemming como alternativa final orientada a un mejor equilibrio entre recuperación y precisión, con recall macro de 0.8600, precisión macro de 0.7493 y F1 macro de 0.7952.

La segunda línea de trabajo analiza comentarios del ámbito hospitalario. Sobre 177.530 comentarios válidos se extrajo una muestra estratificada de 40.000 y se construyeron cinco variantes de preprocesamiento. Las representaciones TF-IDF y Word2Vec se evaluaron con SVM en modo supervisado con solo el 10 % de las etiquetas disponibles (semilla) y en modo semi-supervisado mediante self-training. En total se compararon 20 configuraciones. La mejor configuración en self-training correspondió a TF-IDF con stemming y SVM, con F1 macro de 0.666. Ajustando hiperparámetros sobre el conjunto de entrenamiento etiquetado, el modelo final alcanzó F1 macro de 0.959 y exactitud de 0.995 en prueba.

Índice

1. Introducción y descripción del problema	4
2. Organización del proyecto y alcance de las líneas de trabajo	4
3. Datasets utilizados	5
3.1. Línea A: conjunto de comentarios del sector seguros	5
3.2. Línea B: conjunto de comentarios del ámbito hospitalario	6
4. Descripción de la solución implementada	6
4.1. Arquitectura general	6
4.2. Módulos principales y decisiones de diseño	7
4.3. Tecnologías, librerías y herramientas	7
5. Preparación y preprocesamiento de los comentarios	8
5.1. Línea A: limpieza y normalización	8
5.2. Línea A: variantes de texto evaluadas	8
5.3. Línea A: taxonomía y etiquetado multietiqueta	8
5.4. Línea B: limpieza y normalización	9
5.5. Línea B: variantes de texto evaluadas	9
5.6. Línea B: taxonomía y etiquetado	10
6. Representaciones textuales	10
6.1. Línea A: TF-IDF con n-gramas	10
6.2. Línea A: Word2Vec promedio	11
6.3. Línea B: parámetros y resultados de las representaciones	11
7. Metodología experimental	12
7.1. Criterios comunes de comparación	12
7.2. Línea A: diseño de los experimentos	12
7.3. Línea A: división de entrenamiento, validación y prueba	12
7.4. Línea A: clasificador KNN	13
7.5. Línea A: búsqueda de hiperparámetros	13
7.6. Línea B: diseño de los experimentos	14
7.7. Línea B: división del pool etiquetado	15
7.8. Línea B: clasificador SVM	15
7.9. Línea B: búsqueda de hiperparámetros	15

8. Métricas de evaluación	16
9. Resultados experimentales	16
9.1. Línea A: hiperparámetros seleccionados	16
9.2. Línea A: comparación global de los diez experimentos antes del ajuste de umbrales	17
9.3. Línea A: mejor resultado observado antes del ajuste de umbrales	17
9.4. Línea A: reporte de clasificación del mejor modelo antes del ajuste de umbrales	18
9.5. Etapa KNN 7: Ajuste de umbrales por categoría para los 10 modelos	19
9.6. Línea B: comparación global de las 20 configuraciones	20
9.7. Línea B: mejor resultado observado	20
9.8. Línea B: modelo supervisado de referencia y reporte de clasificación	21
9.9. Síntesis comparativa entre las líneas de trabajo	21
10. Análisis crítico de resultados	21
10.1. Línea A: comparación entre TF-IDF con n-gramas y Word2Vec	21
10.2. Línea A: efecto del preprocesamiento	22
10.3. Línea A: interpretación por categoría	22
10.4. Línea A: efecto del ajuste de umbrales	23
10.5. Línea A: efecto de los hiperparámetros y tiempos	23
10.6. Línea B: comparación entre TF-IDF con n-gramas y Word2Vec	23
10.7. Línea B: efecto del preprocesamiento	24
10.8. Línea B: interpretación por categoría	24
10.9. Línea B: efecto del self-training y los hiperparámetros	24
10.10 Análisis integrado del proyecto	25
10.11 Limitaciones del proyecto	25
11. Conclusiones y trabajo futuro	26
12. Reproducibilidad	27

1. Introducción y descripción del problema

Los comentarios abiertos contienen información relevante sobre la experiencia de usuarios, clientes o pacientes, pero su carácter no estructurado dificulta su análisis sistemático. Para utilizar estos textos en tareas de clasificación, es necesario transformarlos en representaciones numéricas que permitan a los algoritmos identificar patrones y relaciones entre los comentarios. El desempeño de estos enfoques depende, entre otros factores, de la calidad del preprocesamiento, de la representación textual, de la definición de categorías y de la disponibilidad de datos etiquetados [1].

El objetivo general del proyecto es comparar el comportamiento de modelos de clasificación sobre dos conjuntos de comentarios abiertos analizados de manera independiente. La comparación considera el efecto de distintas decisiones de preprocesamiento, representaciones numéricas, configuraciones de hiperparámetros y métricas de desempeño. El trabajo no busca establecer una regla universal acerca de qué representación es superior; busca generar evidencia experimental situada en los corpus, taxonomías, muestras etiquetadas y configuraciones implementadas por cada integrante.

La primera línea de trabajo presenta resultados completos con KNN sobre el conjunto de comentarios del sector seguros. La segunda línea sigue una estructura metodológica paralela con SVM sobre un conjunto de datos del ámbito hospitalario. Esta organización permite integrar resultados de dominios textuales distintos sin asumir que sus distribuciones, etiquetas o configuraciones óptimas sean necesariamente equivalentes.

2. Organización del proyecto y alcance de las líneas de trabajo

El proyecto contempla dos conjuntos de datos y dos líneas de trabajo desarrolladas en paralelo. Cada línea contiene su propio proceso de preparación de datos, representación textual, entrenamiento, ajuste de hiperparámetros, evaluación y análisis de resultados. En consecuencia, las métricas y conclusiones deben interpretarse primero dentro de cada conjunto de datos y, posteriormente, compararse solo cuando las definiciones de las tareas y los criterios de evaluación lo permitan.

Cuadro 1: Organización general de las líneas de trabajo.

Línea de trabajo	Responsable	Alcance
Línea A: comentarios del sector seguros	Gustavo Montoya	Procesamiento de cuatro preguntas abiertas, generación de cinco variantes textuales, representación con TF-IDF y Word2Vec, clasificación multietiqueta KNN, ajuste de hiperparámetros y evaluación experimental.
Línea B: comentarios del ámbito hospitalario	Franco Reinoso	Procesamiento de 177.530 comentarios hospitalarios, generación de cinco variantes textuales, representación con TF-IDF y Word2Vec, clasificación semi-supervisada con SVM mediante self-training, ajuste de hiperparámetros y evaluación experimental sobre 20 configuraciones.

3. Datasets utilizados

3.1. Línea A: conjunto de comentarios del sector seguros

La Línea A utiliza comentarios provenientes de una encuesta de satisfacción del sector seguros. La base original contiene 13.113 registros y 233 columnas. Para el análisis textual se seleccionaron las preguntas abiertas Q2_10_TEXT, Q3, Q4 y Q5. Antes de la reestructuración, estas columnas contenían 1.956, 1.634, 339 y 2.190 respuestas no nulas, respectivamente.

La base fue transformada desde formato ancho a formato largo mediante la función `melt` de `pandas`, generando una fila por comentario potencial. A partir de 52.452 combinaciones iniciales, se eliminaron valores nulos, celdas vacías y textos residuales como `nan`. Como resultado, se obtuvo un universo de 6.119 comentarios válidos para el procesamiento posterior.

Cuadro 2: Resumen del conjunto de datos de la Línea A.

Elemento	Resultado
Base original	13.113 registros y 233 columnas.
Preguntas abiertas utilizadas	Q2_10_TEXT, Q3, Q4 y Q5.
Combinaciones iniciales luego de <code>melt</code>	52.452 filas potenciales.
Comentarios válidos después de limpieza inicial	6.119 comentarios.
Comentarios etiquetados manualmente	1.500 comentarios.
Tipo de problema	Clasificación multietiqueta con cinco categorías temáticas.

3.2. Línea B: conjunto de comentarios del ámbito hospitalario

La Línea B utiliza comentarios de una encuesta de satisfacción del ámbito hospitalario. La base original contiene 252.396 registros y 14 columnas. Para el análisis se seleccionaron la columna `comentario`, la calificación global `calificacion_IBB` y cinco columnas binarias de tópico: `atencion`, `espera`, `pago`, `agendamiento` e `infraestructura`.

La limpieza eliminó artefactos de exportación Excel, normalizó el texto, descartó comentarios con menos de cuatro palabras reales y eliminó duplicados. Tras este proceso se obtuvieron 177.530 comentarios válidos. Para mantener la carga computacional acotada se extrajo una muestra estratificada de 40.000 comentarios según la distribución de `calificacion_IBB`.

Para el experimento semi-supervisado se utilizaron únicamente los comentarios con exactamente un tópico activo, formando un pool multiclase de 21.998 registros distribuidos en cinco categorías: `atencion` (19.221 casos), `espera` (1.778), `pago` (481), `infraestructura` (416) y `agendamiento` (102). Este subconjunto presenta un desbalance marcado, con la categoría `atencion` representando el 87 % de los casos.

Cuadro 3: Resumen del conjunto de datos de la Línea B.

Elemento	Resultado
Base original	252.396 registros y 14 columnas.
Columnas textuales utilizadas	<code>comentario</code> , <code>calificacion_IBB</code> y cinco columnas de tópico (<code>atencion</code> , <code>espera</code> , <code>pago</code> , <code>agendamiento</code> , <code>infraestructura</code>).
Comentarios válidos después de limpieza	177.530 comentarios tras limpieza y eliminación de duplicados.
Comentarios utilizados en experimento	21.998 con exactamente un tópico (pool multiclase); semilla etiquetada: 1.759.
Tipo de problema	Clasificación multiclase de cinco categorías con escenario semi-supervisado (self-training).

4. Descripción de la solución implementada

4.1. Arquitectura general

La solución global se implementó como dos pipelines reproducibles y paralelos. Aunque cada línea trabaja con un corpus propio, ambas comparten una lógica general: ingesta y estructuración de datos, limpieza y normalización, generación de variantes textuales, extracción de características, construcción de datos etiquetados, entrenamiento, ajuste de hiperparámetros, evaluación y análisis crítico.

Dataset A → preprocesamiento A → representaciones A → modelos A → evaluación A

Dataset B → preprocesamiento B → representaciones B → modelos B → evaluación B

Resultados por línea → síntesis comparativa del proyecto

4.2. Módulos principales y decisiones de diseño

Cuadro 4: Módulos principales de los pipelines implementados.

Módulo	Función principal	Decisión de diseño relevante
Estructuración de comentarios	Transformar las fuentes originales a una estructura apta para procesamiento textual.	La separación por línea permite adaptar el tratamiento a las particularidades de cada fuente.
Limpieza textual	Eliminar ruido técnico y normalizar el contenido.	Se mantiene una estrategia conservadora para evitar eliminar señales semánticas potencialmente útiles.
Preprocesamiento experimental	Crear versiones alternativas del mismo comentario.	Las variantes se evalúan separadamente para atribuir diferencias de desempeño a decisiones específicas de preprocesamiento.
Extracción de características	Transformar texto en vectores numéricos.	Se comparan representaciones alternativas de acuerdo con el diseño experimental de cada línea.
Etiquetado y trazabilidad	Vincular comentarios, etiquetas y matrices de características.	Cada pipeline conserva identificadores técnicos para asegurar correspondencia fila a fila.
Modelamiento y evaluación	Entrenar, ajustar y comparar configuraciones de clasificación.	Las divisiones train-validation-test se mantienen internamente consistentes dentro de cada línea.

4.3. Tecnologías, librerías y herramientas

Ambas líneas de trabajo se desarrollaron en Python mediante Jupyter Notebook y comparten el mismo conjunto base de librerías: `pandas` y `NumPy` para manipulación de datos; `scikit-learn` para extracción de características, clasificadores, búsqueda de hiperparámetros y métricas de evaluación; `NLTK` para stopwords y stemming mediante `SnowballStemmer`; `spaCy` con el modelo `es_core_news_sm` para lematización; y `Gensim` para Word2Vec [2].

La diferencia entre líneas reside en los componentes de `scikit-learn` utilizados. La Línea A empleó `KNeighborsClassifier` para clasificación multietiqueta. La Línea B empleó `LinearSVC` con calibración de probabilidad (`CalibratedClassifierCV`) y `SelfTrainingClassifier` para el esquema semi-supervisado, además de `GridSearchCV` para el ajuste de hiperparámetros.

5. Preparación y preprocesamiento de los comentarios

5.1. Línea A: limpieza y normalización

El preprocesamiento de la Línea A fue conservador. Se normalizaron caracteres Unicode, se transformaron los textos a minúsculas, se eliminaron etiquetas HTML, se reemplazaron URLs, correos electrónicos, teléfonos y números largos por tokens genéricos, se corrigieron saltos de línea y tabulaciones, se redujeron repeticiones excesivas de caracteres y se eliminaron símbolos no informativos. Adicionalmente, se aplicó un diccionario de abreviaciones para normalizar expresiones como `pq`, `xq`, `q`, `tb`, `tbn` y `hrs`.

5.2. Línea A: variantes de texto evaluadas

Se construyeron cinco versiones de cada comentario para analizar el efecto del preprocesamiento sobre la clasificación. Estas variantes se presentan en la Tabla 5.

Cuadro 5: Versiones de texto evaluadas en la Línea A.

Variable	Descripción
<code>texto_v1_limpio</code>	Texto con limpieza conservadora, manteniendo tildes, stopwords y la estructura básica del comentario.
<code>texto_v2_sin_tildes</code>	Versión equivalente al texto limpio, pero sin tildes y preservando la letra <code>Ã</code> .
<code>texto_v3_sin_stopwords</code>	Texto sin stopwords controladas. Se conservan negaciones como <code>no</code> , <code>sin</code> , <code>nunca</code> , <code>ni</code> y <code>tampoco</code> .
<code>texto_v4_stemming</code>	Texto transformado mediante stemming en español con <code>SnowballStemmer</code> .
<code>texto_v5_lemma</code>	Texto lematizado con <code>spaCy</code> , reduciendo las palabras a su forma base.

La eliminación de stopwords se aplicó de manera controlada porque, en comentarios breves, las negaciones pueden modificar completamente el significado. Expresiones como “no informaron”, “sin respuesta” o “nunca llegó” perderían parte de su sentido si se eliminan estas palabras.

5.3. Línea A: taxonomía y etiquetado multietiqueta

Se etiquetaron manualmente 1.500 comentarios mediante cinco categorías temáticas. El problema se definió como multietiqueta, de modo que un comentario puede activar más de una categoría simultáneamente. En consecuencia, la suma de etiquetas positivas supera el número total de comentarios etiquetados.

Cuadro 6: Distribución de categorías en la muestra etiquetada de la Línea A.

Categoría	Asignaciones positivas
cat_Pago_monto_y_calculo	343
cat_Tiempos_y_oportunidad	339
cat_Informacion_y_comunicacion	327
cat_Atencion_y_gestion_del_servicio	320
cat_Satisfaccion_o_comentario_general	300
Total de asignaciones positivas	1.629

La muestra de la Línea A contiene 1.378 comentarios con una sola etiqueta, 116 con dos etiquetas, 5 con tres etiquetas y 1 con cuatro etiquetas. Esta distribución confirma que el problema no puede modelarse adecuadamente como clasificación multiclase exclusiva.

5.4. Línea B: limpieza y normalización

La estrategia de limpieza de la Línea B es equivalente a la de la Línea A: normalización Unicode, conversión a minúsculas, eliminación de etiquetas HTML, sustitución de URLs, correos electrónicos, teléfonos y números largos por tokens genéricos, reducción de repeticiones excesivas de caracteres, eliminación de símbolos no informativos y aplicación del mismo diccionario de abreviaciones.

Las diferencias respecto a la Línea A son dos. Primero, el dataset hospitalario provenía de un archivo Excel que introducía el artefacto `_x000D_` en los saltos de línea; este artefacto debió eliminarse como paso previo antes de aplicar cualquier otra normalización. Segundo, a diferencia de la Línea A (que partió de cuatro preguntas abiertas en formato ancho y las unificó mediante un `melt`), la Línea B contaba con una única columna `comentario` y no requirió reestructuración. En su lugar, se aplicó un filtro de contenido mínimo, descartando comentarios con menos de cuatro palabras reales y eliminando duplicados exactos. Tras este proceso, de los 252.396 registros originales se obtuvieron 177.530 comentarios válidos.

5.5. Línea B: variantes de texto evaluadas

Se construyeron las mismas cinco variantes de texto que en la Línea A, descritas en la Tabla 7. Las variantes de stemming y lematización se generaron sobre la muestra de 40.000 comentarios para mantener la carga computacional acotada.

Cuadro 7: Versiones de texto evaluadas en la Línea B.

Variable	Descripción
<code>comentario_limpio</code>	Texto con limpieza conservadora, manteniendo tildes, stopwords y la estructura básica del comentario.
<code>sin_tildes</code>	Versión equivalente al texto limpio, pero sin tildes y preservando la letra Ñ.
<code>sin_stopwords</code>	Texto sin stopwords controladas. Se conservan negaciones como <code>no</code> , <code>sin</code> , <code>nunca</code> , <code>ni</code> y <code>tampoco</code> .
<code>stemming</code>	Texto transformado mediante stemming en español con <code>SnowballStemmer</code> .
<code>lematizado</code>	Texto lematizado con <code>spaCy</code> , reduciendo las palabras a su forma base.

5.6. Línea B: taxonomía y etiquetado

El etiquetado no fue manual: las cinco columnas binarias de tópico ya presentes en el dataset se usaron como verdad de terreno. Para el experimento semi-supervisado se seleccionaron únicamente los registros con exactamente un tópico activo, definiendo un problema de clasificación multiclase de cinco categorías. El subconjunto resultante, denominado pool multiclase, contiene 21.998 comentarios. La distribución por categoría se presenta en la Tabla 8.

Cuadro 8: Distribución de categorías en el pool multiclase de la Línea B.

Categoría	Comentarios
<code>atencion</code>	19.221
<code>espera</code>	1.778
<code>pago</code>	481
<code>infraestructura</code>	416
<code>agendamiento</code>	102
Total pool multiclase	21.998

La distribución muestra un desbalance marcado: la categoría `atencion` concentra el 87 % de los casos. Este desbalance es relevante para la interpretación del F1 macro y para el comportamiento del self-training en las clases minoritarias.

6. Representaciones textuales

6.1. Línea A: TF-IDF con n-gramas

TF-IDF transforma cada comentario en un vector disperso que representa la importancia relativa de palabras y secuencias de palabras dentro del corpus. Se utilizaron unigramas y bigramas mediante `ngram_range=(1, 2)`, permitiendo conservar tanto términos individuales como expresiones compuestas, por ejemplo, “pago pendiente”, “sin respuesta” o “monto incorrecto”.

La configuración utilizada fue `min_df=2`, `max_df=0.95`, `sublinear_tf=True`, `lowercase=False` y tipo de dato `float32`. El parámetro `lowercase=False` se utilizó porque

la conversión a minúsculas ya había sido realizada durante la limpieza previa. En la representación inicial del universo completo, TF-IDF produjo una matriz dispersa de 6.119 filas y 7.110 características.

6.2. Línea A: Word2Vec promedio

Word2Vec genera vectores densos para las palabras a partir de sus contextos de aparición. Se utilizó Skip-gram con vectores de 100 dimensiones, ventana de contexto igual a 5, `min_count=2` y semilla 42. El vocabulario inicial generado para el corpus completo incluyó 2.193 términos.

Word2Vec produce vectores a nivel de palabra, mientras que KNN requiere una representación única por comentario. Por esta razón, el vector de cada comentario se construyó mediante el promedio de los embeddings de las palabras presentes en su vocabulario. Esta decisión genera una representación densa de dimensión fija, aunque puede perder información sobre el orden de las palabras y sobre expresiones locales relevantes.

Cuadro 9: Configuración de las representaciones textuales de la Línea A.

Representación	Configuración	Salida
TF-IDF con n-gramas	Unigramas y bigramas; <code>min_df=2</code> ; <code>max_df=0.95</code> ; <code>sublinear_tf=True</code> .	Matriz dispersa de características léxicas.
Word2Vec	Skip-gram; 100 dimensiones; ventana 5; <code>min_count=2</code> ; promedio de embeddings por comentario.	Matriz densa de 100 dimensiones por comentario.

6.3. Línea B: parámetros y resultados de las representaciones

La Línea B aplicó las mismas dos representaciones descritas en las secciones anteriores. Los parámetros difieren en dos aspectos respecto a la Línea A, ambos justificados por el mayor tamaño del corpus. En TF-IDF, `min_df` se fijó en 5 (frente a 2 en la Línea A) para descartar términos muy poco frecuentes en un vocabulario más amplio, y se agregó `norm="l2"` para normalizar los vectores. En Word2Vec, `min_count` se fijó en 3 (frente a 2) por la misma razón. El resto de la configuración es idéntica: unigramas y bigramas, Skip-gram con 100 dimensiones y ventana 5, y representación por comentario mediante promedio de embeddings. La Tabla 10 resume los parámetros y las dimensiones de salida obtenidas.

Cuadro 10: Configuración y salida de las representaciones textuales de la Línea B.

Representación	Configuración	Salida
TF-IDF con n-gramas	<code>min_df=5; max_df=0.95; sublinear_tf=True; norm="l2"</code> (resto igual a Línea A).	Matriz dispersa de 40.000×22.585 características.
Word2Vec	<code>min_count=3; epochs=10</code> (resto igual a Línea A).	Matriz densa de 40.000×100 ; vocabulario de 8.569 términos.

7. Metodología experimental

7.1. Criterios comunes de comparación

Las evaluaciones se diseñaron para comparar configuraciones dentro de cada línea de trabajo. Cada pipeline mantiene separadas sus propias observaciones, etiquetas, representaciones y particiones de datos. Esta decisión evita transferir conclusiones de un dominio a otro sin evidencia empírica y permite que las comparaciones entre líneas se realicen únicamente a nivel metodológico o mediante métricas efectivamente comparables.

7.2. Línea A: diseño de los experimentos

En la Línea A se construyeron diez experimentos independientes, combinando las cinco versiones de texto con las dos representaciones numéricas. No se concatenaron las versiones de texto en una misma matriz, ya que el objetivo fue medir de manera aislada el efecto de cada decisión de preprocesamiento.

Cuadro 11: Diseño experimental de la Línea A.

Grupo	Representación	Versiones evaluadas
1–5	TF-IDF con n-gramas	Limpio, sin tildes, sin stopwords, stemming y lematización.
6–10	Word2Vec promedio	Limpio, sin tildes, sin stopwords, stemming y lematización.

Todos los experimentos de esta línea utilizaron la misma semilla, las mismas categorías objetivo y la misma división train-validation-test. Esto permite atribuir las diferencias observadas principalmente a la representación textual, la variante de preprocesamiento y la configuración KNN seleccionada.

7.3. Línea A: división de entrenamiento, validación y prueba

La muestra etiquetada se dividió mediante estratificación multietiqueta. El objetivo fue mantener, en lo posible, la presencia relativa de cada categoría en los subconjuntos. La partición utilizada se presenta en la Tabla 12.

Cuadro 12: División de los comentarios etiquetados de la Línea A.

Partición	Comentarios	Proporción aproximada
Entrenamiento	1.051	70 %
Validación	226	15 %
Prueba	223	15 %
Total	1.500	100 %

Para evitar fuga de información, los vectores TF-IDF y los modelos Word2Vec utilizados durante la validación fueron ajustados únicamente con los comentarios de entrenamiento. Después de seleccionar la mejor configuración KNN de cada experimento con el conjunto de validación, la representación se volvió a ajustar usando entrenamiento más validación. El conjunto de prueba se mantuvo aislado hasta la evaluación final.

7.4. Línea A: clasificador KNN

KNN es un algoritmo no paramétrico basado en instancias. En lugar de estimar coeficientes mediante la optimización de una función de pérdida, el algoritmo almacena los ejemplos de entrenamiento y clasifica un ejemplo nuevo según sus vecinos más cercanos en el espacio de características. Por esta razón, KNN no utiliza función de costo, descenso de gradiente ni épocas de entrenamiento.

En la Línea A, KNN se utilizó en un escenario multietiqueta, donde cada variable `cat_` representa una salida binaria. El tiempo de ajuste suele ser bajo porque la etapa `fit` almacena los ejemplos de referencia; en cambio, el tiempo de predicción es especialmente relevante, pues el algoritmo debe calcular distancias entre los comentarios nuevos y los ejemplos almacenados.

La distancia coseno utilizada en parte de las configuraciones se expresa como:

$$d_{\text{coseno}}(\mathbf{x}, \mathbf{y}) = 1 - \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}. \quad (1)$$

Para las configuraciones Minkowski se utilizó:

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{j=1}^m |x_j - y_j|^p \right)^{\frac{1}{p}}, \quad (2)$$

donde $p = 1$ corresponde a distancia Manhattan y $p = 2$ a distancia euclidiana.

7.5. Línea A: búsqueda de hiperparámetros

Se evaluaron los valores `n_neighbors` iguales a 3, 5, 7, 9 y 11; ponderación uniforme y por distancia; métrica coseno y Minkowski; valores $p = 1$ y $p = 2$ para Minkowski; y distintas estrategias de búsqueda para las configuraciones Word2Vec.

Cuadro 13: Espacio de búsqueda de hiperparámetros KNN en la Línea A.

Parámetro	Valores evaluados	Interpretación
<code>n_neighbors</code>	3, 5, 7, 9 y 11	Cantidad de vecinos utilizados en la decisión de clasificación.
<code>weights</code>	<code>uniform</code> , <code>distance</code>	Votación equivalente o ponderada por cercanía.
<code>metric</code>	<code>cosine</code> , <code>minkowski</code>	Medida de similitud o distancia entre vectores.
<code>p</code>	1 y 2	Parámetro de Minkowski: Manhattan y euclidiana, respectivamente.
<code>algorithm</code>	<code>auto</code> , <code>ball_tree</code> , <code>kd_tree</code> , <code>brute</code>	Estrategia de búsqueda de vecinos.

Para TF-IDF se utilizó búsqueda `brute`, ya que esta representación es dispersa y de alta dimensionalidad. Se evaluaron 30 configuraciones por cada variante TF-IDF y 90 configuraciones por cada variante Word2Vec, totalizando 600 configuraciones KNN. La configuración ganadora de cada experimento fue seleccionada mediante F1 macro sobre el conjunto de validación. En caso de empate, se priorizó F1 micro, precisión macro y menor tiempo de predicción.

7.6. Línea B: diseño de los experimentos

Se construyeron 20 experimentos combinando las cinco variantes de texto, las dos representaciones y los dos modos de entrenamiento (supervisado y self-training), con SVM como clasificador único. No se concatenaron variantes en una misma matriz; el objetivo fue medir de manera aislada el efecto de cada decisión de preprocesamiento. El diseño se presenta en la Tabla 14.

Cuadro 14: Diseño experimental de la Línea B.

Grupo	Representación	Versiones evaluadas
1–5	TF-IDF con n-gramas	Limpio, sin tildes, sin stopwords, stemming y lematizado.
6–10	Word2Vec promedio	Limpio, sin tildes, sin stopwords, stemming y lematizado.

Cada uno de los diez grupos se evaluó con un clasificador (SVM) y en dos modos (supervisado con solo la semilla y self-training), resultando en 20 configuraciones totales. Todos los experimentos utilizaron la misma semilla aleatoria, el mismo pool multiclase y la misma partición train–test.

7.7. Línea B: división del pool etiquetado

El pool multiclase se dividió mediante estratificación por categoría para mantener la distribución relativa en cada subconjunto. A diferencia de la Línea A, no se usó un conjunto de validación independiente: la comparación entre las 20 configuraciones se realizó directamente sobre el conjunto de prueba. La partición se presenta en la Tabla 15.

Cuadro 15: División del pool multiclase de la Línea B.

Partición	Comentarios	Proporción
Entrenamiento — semilla etiquetada	1.759	10 % del entrenamiento
Entrenamiento — no etiquetado	15.839	90 % del entrenamiento
Prueba (etiquetada, reservada)	4.400	20 % del pool total
Total pool multiclase	21.998	100 %

El conjunto de prueba se mantuvo aislado en todo momento. Las etiquetas del conjunto de entrenamiento no semilla fueron ocultadas durante la fase de self-training, revelándose solo para calcular las métricas finales.

7.8. Línea B: clasificador SVM

Se utilizó SVM con calibración de probabilidad (`CalibratedClassifierCV(Linear SVC)`), que genera puntuaciones de confianza necesarias para el criterio de umbral del self-training.

El self-training (`SelfTrainingClassifier`, umbral 0.80) utiliza el clasificador base para etiquetar iterativamente los ejemplos no etiquetados cuya probabilidad predicha supera ese umbral, reentrenando el modelo en cada iteración hasta que no quedan ejemplos por etiquetar o se alcanza la convergencia.

7.9. Línea B: búsqueda de hiperparámetros

Sobre la mejor configuración en self-training se realizó una búsqueda exhaustiva de hiperparámetros con `GridSearchCV` y validación cruzada de 3 folds. El espacio de búsqueda se detalla en la Tabla 16.

Cuadro 16: Espacio de búsqueda de hiperparámetros en la Línea B.

Clasificador	Parámetros evaluados	Interpretación
SVM (<code>Linear SVC</code>)	$C \in \{0.1, 1, 10\}$	Parámetro de regularización: valores altos penalizan menos el margen.

La búsqueda se realizó solo sobre la mejor variante encontrada en la comparación de 20 configuraciones, utilizando como criterio F1 macro. El conjunto de prueba no participó en este proceso.

8. Métricas de evaluación

Las evaluaciones realizadas utilizan métricas adecuadas para clasificación multietiqueta cuando la estructura de las etiquetas de cada línea de trabajo lo requiere. Se consideran precisión, recall y F1 por categoría, junto con promedios macro, micro y ponderados. La precisión se define como:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (3)$$

donde TP representa verdaderos positivos y FP falsos positivos. El recall se define como:

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (4)$$

donde FN representa falsos negativos. El F1-score combina ambas medidas:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (5)$$

En la Línea A, F1 macro se utilizó como métrica principal para seleccionar hiperparámetros, ya que asigna el mismo peso a cada categoría y evita que las etiquetas más frecuentes dominen el criterio de selección. También se reportó la exactitud de subconjunto (*accuracy exacta*), que considera correcta una predicción solo cuando coincide el conjunto completo de etiquetas de un comentario.

Adicionalmente, se utilizó Hamming loss, que mide la proporción de decisiones de etiqueta incorrectas respecto del total de decisiones posibles. Esta métrica permite cuantificar errores parciales en problemas multietiqueta, incluso cuando la totalidad de etiquetas de un comentario no coincide exactamente.

Para la Línea B las métricas utilizadas fueron exactitud, F1 macro y F1 ponderado sobre el conjunto de prueba de 4.400 comentarios. La métrica principal para comparar las 20 configuraciones fue F1 macro, que asigna el mismo peso a cada categoría independientemente de su frecuencia. Dado el desbalance marcado del pool multiclase (**atencion** concentra el 87 % de los casos), el F1 macro es más informativo que la exactitud para evaluar el comportamiento en clases minoritarias. Para el modelo final se reportaron también precisión, recall y F1-score por clase mediante `classification_report`.

9. Resultados experimentales

9.1. Línea A: hiperparámetros seleccionados

La Tabla 17 presenta la mejor configuración KNN seleccionada para cada una de las diez combinaciones de representación y preprocesamiento de la Línea A. La selección fue realizada con el conjunto de validación antes de la medición final en prueba.

Cuadro 17: Configuraciones KNN seleccionadas para la Línea A.

N°	Representación	Versión de texto	k	Pesos	Métrica	p	Algoritmo
1	TF-IDF n-gramas	texto_v1_limpio	7	distance	cosine	–	brute
2	TF-IDF n-gramas	texto_v2_sin_tildes	7	distance	cosine	–	brute
3	TF-IDF n-gramas	texto_v3_sin_stopwords	3	uniform	cosine	–	brute
4	TF-IDF n-gramas	texto_v4_stemming	3	uniform	cosine	–	brute
5	TF-IDF n-gramas	texto_v5_lemma	5	distance	minkowski	2	brute
6	Word2Vec	texto_v1_limpio	7	uniform	cosine	–	brute
7	Word2Vec	texto_v2_sin_tildes	3	distance	cosine	–	brute
8	Word2Vec	texto_v3_sin_stopwords	5	distance	cosine	–	brute
9	Word2Vec	texto_v4_stemming	3	distance	cosine	–	brute
10	Word2Vec	texto_v5_lemma	3	distance	cosine	–	brute

9.2. Línea A: comparación global de los diez experimentos antes del ajuste de umbrales

La Tabla 18 presenta los resultados base sobre el conjunto de prueba de la Línea A, antes del ajuste de umbrales por categoría. Las configuraciones están ordenadas de mayor a menor F1 macro. Los tiempos de predicción corresponden a la ejecución final de KNN sobre 223 comentarios de prueba; no incluyen el tiempo utilizado en la construcción de la representación ni la búsqueda previa de hiperparámetros.

Cuadro 18: Resultados base de los experimentos KNN de la Línea A antes del ajuste de umbrales.

Rank	Representación	Versión	Accuracy	Prec. macro	Recall macro	F1 macro	F1 micro	Hamming loss	Pred. (s)
1	TF-IDF n-gramas	texto_v5_lemma	0.7265	0.9177	0.7377	0.8133	0.8182	0.0717	0.0995
2	TF-IDF n-gramas	texto_v4_stemming	0.7354	0.8713	0.7543	0.8051	0.8053	0.0798	0.0894
3	TF-IDF n-gramas	texto_v3_sin_stopwords	0.7130	0.8730	0.7430	0.7958	0.7956	0.0834	0.0812
4	TF-IDF n-gramas	texto_v2_sin_tildes	0.6906	0.9183	0.7051	0.7935	0.7963	0.0789	0.0894
5	TF-IDF n-gramas	texto_v1_limpio	0.6951	0.9132	0.7051	0.7900	0.7926	0.0807	0.0952
6	Word2Vec	texto_v4_stemming	0.7354	0.8299	0.7498	0.7825	0.7854	0.0897	0.0929
7	Word2Vec	texto_v3_sin_stopwords	0.5830	0.7790	0.6071	0.6813	0.6805	0.1247	0.0875
8	Word2Vec	texto_v5_lemma	0.5830	0.7450	0.6205	0.6701	0.6756	0.1309	0.0991
9	Word2Vec	texto_v1_limpio	0.5471	0.8383	0.5489	0.6410	0.6505	0.1291	0.1003
10	Word2Vec	texto_v2_sin_tildes	0.5471	0.7023	0.5859	0.6253	0.6258	0.1534	0.0926

Las cinco configuraciones TF-IDF ocuparon las primeras cinco posiciones del ranking por F1 macro. Este resultado indica que, bajo las condiciones experimentales de la Línea A, las características léxicas representadas mediante unigramas y bigramas ofrecieron mayor capacidad discriminativa que Word2Vec promedio.

9.3. Línea A: mejor resultado observado antes del ajuste de umbrales

Antes del ajuste de umbrales por categoría, el mejor resultado observado en prueba para la Línea A correspondió a KNN__TFIDF_NGRAMS__texto_v5_lemma. Esta configuración utilizó texto lematizado, TF-IDF con unigramas y bigramas, cinco vecinos, ponderación por distancia y métrica Minkowski con $p = 2$, equivalente a distancia euclidiana.

Cuadro 19: Métricas del mejor resultado observado en la Línea A antes del ajuste de umbrales.

Métrica	Valor
Accuracy exacta	0.7265
Precision macro	0.9177
Recall macro	0.7377
F1 macro	0.8133
Precision micro	0.9184
Recall micro	0.7377
F1 micro	0.8182
F1 ponderado	0.8143
Hamming loss	0.0717
Tiempo de ajuste KNN	0.0012 s
Tiempo de predicción en prueba	0.0995 s

La exactitud exacta de 0.7265 indica que aproximadamente el 72.65 % de los comentarios de prueba de la Línea A recibió exactamente el mismo conjunto de etiquetas que la anotación manual. Esta es una métrica estricta, ya que basta una etiqueta incorrecta, omitida o adicional para que el comentario sea considerado incorrecto en esta medida.

9.4. Línea A: reporte de clasificación del mejor modelo antes del ajuste de umbrales

La Tabla 20 presenta el desempeño por categoría del mejor resultado observado antes del ajuste de umbrales. El soporte corresponde a la cantidad de asignaciones positivas de cada categoría en el conjunto de prueba. Su suma es superior al número de comentarios de prueba debido a la naturaleza multietiqueta del problema.

Cuadro 20: Reporte de clasificación del mejor resultado observado en la Línea A antes del ajuste de umbrales.

Categoría	Precision	Recall	F1-score	Support
Pago, monto y cálculo	0.9545	0.8235	0.8842	51
Tiempos y oportunidad	1.0000	0.7843	0.8791	51
Información y comunicación	0.8966	0.5306	0.6667	49
Atención y gestión del servicio	0.9000	0.7500	0.8182	48
Satisfacción o comentario general	0.8372	0.8000	0.8182	45
Micro promedio	0.9184	0.7377	0.8182	244
Macro promedio	0.9177	0.7377	0.8133	244
Ponderado promedio	0.9200	0.7377	0.8143	244

9.5. Etapa KNN 7: Ajuste de umbrales por categoría para los 10 modelos

La etapa base utilizó la decisión estándar de KNN para asignar cada categoría. Sin embargo, un único criterio de decisión no necesariamente es adecuado para todas las etiquetas, ya que cada categoría presenta distinta frecuencia, separabilidad y relación entre precisión y recall. Por ello, se realizó una calibración independiente por categoría para los diez modelos KNN evaluados.

Para cada modelo se obtuvieron probabilidades de clase sobre el conjunto de validación y se evaluaron umbrales candidatos entre 0.05 y 0.95, con incrementos de 0.05. El umbral de cada categoría se seleccionó maximizando F_2 , métrica que otorga mayor importancia al recall. En caso de empate se priorizaron, sucesivamente, recall, F1, precisión y cercanía al umbral estándar. Los umbrales seleccionados se mantuvieron fijos y se aplicaron una única vez al conjunto de prueba, el cual no participó en la calibración.

Antes de esta etapa, la configuración TF-IDF con texto lematizado era el mejor modelo según F1 macro de prueba. Después de la calibración, la configuración TF-IDF con stemming fue seleccionada como la alternativa final más equilibrada para un criterio sensible al recall. La selección no implica que su F1 macro sea superior al mejor resultado base; refleja que la calibración permitió recuperar una mayor proporción de etiquetas positivas con una reducción controlada de precisión.

Cuadro 21: Comparación de resultados disponibles antes y después del ajuste de umbrales.

Modelo	Etapas	Prec. macro	Recall macro	F1 macro
TF-IDF + lematización	Mejor resultado base	0.9177	0.7377	0.8133
TF-IDF + stemming	Antes de calibrar	0.8713	0.7543	0.8051
TF-IDF + stemming	Umbrales por categoría	0.7493	0.8600	0.7952

En la misma configuración TF-IDF con stemming, el ajuste elevó el recall macro desde 0.7543 hasta 0.8600, una diferencia de 0.1057. A cambio, la precisión macro disminuyó desde 0.8713 hasta 0.7493 y el F1 macro pasó de 0.8051 a 0.7952. Este comportamiento representa el compromiso esperado al reducir los umbrales de decisión: se detectan más casos positivos, pero aumentan las asignaciones positivas incorrectas.

Cuadro 22: Reporte de clasificación del modelo calibrado seleccionado en la Línea A.

Categoría	Precision	Recall	F1-score	Support
Pago, monto y cálculo	0.7121	0.9216	0.8034	51
Tiempos y oportunidad	0.9070	0.7647	0.8298	51
Información y comunicación	0.7302	0.9388	0.8214	49
Atención y gestión del servicio	0.6774	0.8750	0.7636	48
Satisfacción o comentario general	0.7200	0.8000	0.7579	45
Micro promedio	0.7394	0.8607	0.7955	244
Macro promedio	0.7493	0.8600	0.7952	244
Ponderado promedio	0.7511	0.8607	0.7963	244
Promedio por muestra	0.8042	0.8707	0.8226	244

El modelo calibrado destaca por el recall alcanzado en todas las categorías, particularmente en *Información y comunicación*, donde obtuvo 0.9388. Por tanto, TF-IDF con lematización se mantiene como el mejor modelo de la etapa base según F1 macro, mientras que TF-IDF con stemming se recomienda como configuración final cuando se privilegia una detección más amplia de etiquetas positivas y un equilibrio operativo entre precision y recall.

9.6. Línea B: comparación global de las 20 configuraciones

La Tabla 23 presenta los resultados de las 10 combinaciones únicas (variante $\times$ método), mostrando F1 macro en modo supervisado (solo semilla) y en modo self-training, ordenadas de mayor a menor F1 macro en self-training.

Cuadro 23: Resultados de las configuraciones de la Línea B, ordenadas por F1 macro en self-training.

Rank	Variante	Método	Clf	F1 macro (supervisado)	F1 macro (self-training)	Delta
1	stemming	TF-IDF	SVM	0.709	0.666	−0.043
2	sin_stopwords	TF-IDF	SVM	0.661	0.659	−0.002
3	lematizado	TF-IDF	SVM	0.682	0.615	−0.067
4	stemming	W2V	SVM	0.672	0.585	−0.087
5	sin_stopwords	W2V	SVM	0.620	0.549	−0.071
6	lematizado	W2V	SVM	0.624	0.545	−0.079
7	comentario_limpio	TF-IDF	SVM	0.603	0.517	−0.086
8	sin_tildes	TF-IDF	SVM	0.623	0.514	−0.109
9	sin_tildes	W2V	SVM	0.582	0.513	−0.069
10	comentario_limpio	W2V	SVM	0.599	0.509	−0.090

El self-training produjo resultados inferiores al modo supervisado en las 10 combinaciones, con deltas que van desde −0.002 hasta −0.109 puntos de F1 macro.

9.7. Línea B: mejor resultado observado

El mejor resultado en self-training correspondió a la combinación `stemming|tfidf|svm`. La Tabla 24 resume sus métricas sobre el conjunto de prueba.

Cuadro 24: Métricas del mejor resultado observado en la Línea B en modo self-training.

Métrica	Valor
Configuración	stemming + TF-IDF + SVM
Modo	Self-training (umbral 0.80)
Accuracy	0.966
F1 macro	0.666
F1 ponderado	0.960

La exactitud de 0.966 parece alta, pero refleja el dominio de la clase `atencion` (87 % del pool). El F1 macro de 0.666 es la métrica más representativa del desempeño real sobre las cinco categorías.

9.8. Línea B: modelo supervisado de referencia y reporte de clasificación

Al igual que la Línea A realizó una etapa de calibración de umbrales sobre su mejor modelo base, la Línea B incluye una etapa de referencia supervisada: se entrena el mismo pipeline (stemming + TF-IDF + SVM) con *todas* las etiquetas del conjunto de entrenamiento disponibles, eliminando la restricción del 10 % de semilla. El objetivo no es mejorar el self-training, sino establecer qué tan lejos está el escenario semi-supervisado del techo alcanzable con supervisión completa.

La búsqueda de hiperparámetros seleccionó $C = 10$ con F1 macro de 0.948 en validación cruzada. Sobre el conjunto de prueba, el modelo alcanzó F1 macro de 0.959, frente al 0.666 del mejor resultado en self-training. La diferencia de 0.293 puntos cuantifica el costo de disponer de etiquetas para solo el 10 % del conjunto de entrenamiento en un corpus con desbalance marcado. El reporte completo por categoría se presenta en la Tabla 25.

Cuadro 25: Reporte de clasificación del modelo final de la Línea B (stemming + TF-IDF + SVM, $C = 10$).

Categoría	Precision	Recall	F1-score	Support
agendamiento	1.000	0.800	0.889	20
atencion	0.995	0.999	0.997	3.845
espera	0.994	0.989	0.992	356
infraestructura	0.987	0.904	0.943	83
pago	0.989	0.958	0.974	96
Macro promedio	0.993	0.930	0.959	4.400
Ponderado promedio	0.995	0.995	0.995	4.400

9.9. Síntesis comparativa entre las líneas de trabajo

Las conclusiones comparativas entre ambos conjuntos de datos deben elaborarse únicamente a partir de los resultados, configuraciones y métricas efectivamente obtenidos en cada línea. En particular, no se deben comparar directamente valores de F1, accuracy o tiempos si las taxonomías, tamaños muestrales, particiones o protocolos de evaluación son distintos sin explicitar estas diferencias.

10. Análisis crítico de resultados

10.1. Línea A: comparación entre TF-IDF con n-gramas y Word2Vec

La principal tendencia observada en la Línea A es que TF-IDF con n-gramas superó a Word2Vec en todos los experimentos evaluados según F1 macro. El mejor modelo

TF-IDF alcanzó F1 macro de 0.8133, mientras que el mejor modelo Word2Vec obtuvo 0.7825. Esta diferencia sugiere que, en este corpus, las palabras y expresiones concretas fueron más informativas que las representaciones densas promedio construidas a partir de embeddings de palabras.

Este comportamiento es coherente con comentarios breves vinculados a temas específicos. Expresiones como “monto incorrecto”, “sin respuesta”, “no informaron” o “demora en el pago” pueden ser capturadas directamente por unigramas y bigramas. En Word2Vec, el promedio de embeddings reduce todo el comentario a un único vector denso, lo que puede diluir términos altamente discriminativos y eliminar información sobre el orden de las palabras o sobre combinaciones locales.

No obstante, este resultado no permite afirmar que TF-IDF sea universalmente superior a Word2Vec. La conclusión se restringe al conjunto de datos de la Línea A, al tamaño de su muestra, a la taxonomía, a la partición, al preprocesamiento y al clasificador utilizados.

10.2. Línea A: efecto del preprocesamiento

Dentro de TF-IDF, la lematización alcanzó el mejor F1 macro en prueba, con 0.8133. Este resultado sugiere que agrupar variantes flexivas bajo una forma base puede favorecer la consistencia de la representación. Por ejemplo, expresiones relacionadas con “informar”, “informaron” e “informado” pueden concentrar evidencia en términos más estables.

El stemming fue la segunda mejor configuración TF-IDF, con F1 macro de 0.8051 y la mayor exactitud exacta entre los modelos TF-IDF, igual a 0.7354. La diferencia entre lematización y stemming es acotada, por lo que no se observa una superioridad absoluta de una técnica sobre la otra. En cambio, se observa que ambos mecanismos de reducción morfológica tuvieron un desempeño superior a las variantes limpia, sin tildes y sin stopwords bajo TF-IDF.

Para Word2Vec, la mejor variante fue stemming, con F1 macro de 0.7825. Esto indica que la reducción de variabilidad morfológica también fue favorable para la representación densa, aunque no logró superar a las configuraciones TF-IDF equivalentes.

10.3. Línea A: interpretación por categoría

Las categorías *Pago*, *monto y cálculo* y *Tiempos y oportunidad* alcanzaron los mayores F1-score en el mejor modelo, con 0.8842 y 0.8791, respectivamente. Esto sugiere que estos temas poseen vocabulario relativamente específico y consistente dentro del corpus.

La categoría *Información y comunicación* presentó el menor F1-score, igual a 0.6667. Su precisión fue alta, con 0.8966, pero su recall fue menor, con 0.5306. En términos prácticos, el modelo fue conservador: cuando asignó esta categoría, generalmente acertó, pero dejó sin detectar una proporción relevante de comentarios que sí pertenecían a ella.

Una explicación plausible es que la categoría de información y comunicación comparta vocabulario con atención, tiempos de respuesta o satisfacción general. Comentarios como “no me dieron respuesta”, por ejemplo, pueden aludir simultáneamente a falta de información, atención deficiente y demora. Esta superposición temática dificulta la separación de etiquetas y es consistente con la naturaleza multietiqueta del problema.

10.4. Línea A: efecto del ajuste de umbrales

La calibración por categoría modificó el criterio de selección de la configuración final. El modelo base con texto lematizado mantuvo el mayor F1 macro antes del ajuste, mientras que TF-IDF con stemming fue seleccionado después de calibrar por su recall macro de 0.8600. Respecto de esa misma configuración antes de la calibración, el recall macro aumentó en 0.1057 y el F1 macro disminuyó solo en 0.0099. Esta diferencia evidencia que el ajuste de umbrales no mejora simultáneamente todas las métricas: desplaza el equilibrio hacia una mayor detección de etiquetas positivas.

El resultado es especialmente relevante para categorías con riesgo de falsos negativos. En el modelo calibrado, *Información y comunicación* alcanzó recall de 0.9388, aunque su precisión fue de 0.7302. Por ello, la configuración calibrada se recomienda cuando resulta más importante recuperar comentarios pertinentes que conservar el nivel máximo de precisión obtenido por el modelo base.

10.5. Línea A: efecto de los hiperparámetros y tiempos

Las configuraciones seleccionadas utilizaron principalmente distancia coseno y búsqueda **brute**. La distancia coseno es apropiada para representaciones textuales porque compara la orientación relativa entre vectores, reduciendo el efecto de la longitud absoluta del comentario.

El mejor resultado observado utilizó, en cambio, distancia Minkowski con $p = 2$ y ponderación por distancia. En esta configuración, los vecinos más cercanos influyen más en la decisión que los vecinos lejanos. Sin embargo, no existe una correlación fija entre una métrica de distancia o un valor de k y un mejor desempeño. El efecto de los hiperparámetros depende de la representación, del preprocesamiento y de la distribución de las categorías.

Los tiempos de ajuste fueron muy bajos, del orden de milisegundos. Este comportamiento es esperable en KNN, ya que el algoritmo no ajusta parámetros complejos durante **fit**; principalmente almacena las instancias de referencia. Los tiempos de predicción sobre el conjunto de prueba se ubicaron aproximadamente entre 0.0812 y 0.1003 segundos para 223 comentarios. Por lo tanto, en una eventual aplicación a mayor escala, el tiempo de predicción y la memoria requerida por los vectores de entrenamiento son aspectos más relevantes que el tiempo de ajuste.

10.6. Línea B: comparación entre TF-IDF con n-gramas y Word2Vec

TF-IDF superó a Word2Vec en cuatro de las cinco variantes de texto. En modo self-training, el mejor resultado TF-IDF fue 0.666 (stemming) frente a 0.585 del mejor Word2Vec (también stemming), una diferencia de 0.081 puntos.

Este comportamiento es consistente con la Línea A: las características léxicas de TF-IDF capturan términos y expresiones específicas del corpus hospitalario con mayor efectividad que el promedio de embeddings de Word2Vec. Sin embargo, la conclusión aplica exclusivamente a este corpus, esta taxonomía y estas configuraciones.

10.7. Línea B: efecto del preprocesamiento

Dentro de TF-IDF con SVM, el stemming fue la variante con mayor F1 macro tanto en modo supervisado (0.709) como en self-training (0.666). La lematización fue la segunda mejor variante (supervisado: 0.682, self-training: 0.615), y sin stopwords ocupó el tercer lugar (supervisado: 0.661, self-training: 0.659). Las variantes `comentario_limpio` y `sin_tildes` mostraron consistentemente el menor desempeño.

Para Word2Vec con SVM, el stemming también fue la mejor variante (supervisado: 0.672, self-training: 0.585). La reducción morfológica parece favorecer ambas representaciones, posiblemente porque normaliza variantes flexivas del mismo término, concentrando la evidencia en formas más estables.

10.8. Línea B: interpretación por categoría

Las categorías *atencion* y *espera* alcanzaron los mayores F1-score en el modelo final, con 0.997 y 0.992, respectivamente. Ambas poseen vocabulario específico y suficientes ejemplos de entrenamiento para que el modelo las discrimine bien.

La categoría *agendamiento* presentó el menor F1-score, con 0.889, a pesar de obtener precisión perfecta (1.000). Su recall de 0.800 indica que el modelo dejó sin detectar el 20 % de los casos de prueba. Esto es esperable dado que *agendamiento* cuenta con solo 102 comentarios en todo el pool (20 en prueba), lo que limita la cantidad de señal disponible durante el entrenamiento.

10.9. Línea B: efecto del self-training y los hiperparámetros

La Línea A incluyó una etapa de calibración de umbrales como segundo ajuste: sobre el mismo modelo base, modificó los criterios de decisión sin agregar etiquetas nuevas, logrando subir el recall macro en 0.1057 con una caída de F1 macro de solo 0.0099. El self-training de la Línea B cumple un rol análogo en la estructura del pipeline, pero con un comportamiento opuesto: en lugar de refinar un modelo ya robusto, intenta aprender de datos sin etiquetar, y en ninguna de las 10 combinaciones logró superar al supervisado. La diferencia no es solo de resultados, sino de premisa: la calibración de Línea A parte de un modelo con acceso a todas las etiquetas de entrenamiento; el self-training de Línea B parte de solo el 10 %.

Dentro de ese marco, el menor impacto negativo del self-training se observó en `sin_stopwords|tfidf|svm` (delta -0.002), lo que sugiere que eliminar stopwords reduce la variabilidad del vocabulario y facilita que el modelo propague etiquetas con mayor consistencia. El mayor impacto negativo correspondió a `sin_tildes|tfidf|svm` (delta -0.109): esta variante introduce colisiones léxicas al eliminar las tildes sin reducir vocabulario, lo que genera señales ambiguas durante las iteraciones de self-training y acumula errores de etiquetado progresivos.

Respecto a los hiperparámetros, la búsqueda seleccionó $C = 10$ para SVM, el valor más alto del espacio evaluado. Esto indica que en este corpus, con 17.598 ejemplos de entrenamiento y cinco categorías muy desbalanceadas, el modelo se beneficia de penalizar poco el margen y concentrarse en clasificar correctamente los ejemplos de entrenamiento disponibles.

10.10. Análisis integrado del proyecto

La integración de ambas líneas permite examinar cómo los métodos de procesamiento y clasificación se comportan en dominios distintos. Esta comparación debe considerar las diferencias en tamaño de los corpus, naturaleza de los comentarios, definición de categorías, proporción de etiquetas y representación seleccionada. Por ello, la interpretación integrada no debe reducirse a identificar un único modelo “ganador” para ambos dominios, sino a reconocer las condiciones bajo las cuales cada configuración resultó más adecuada.

10.11. Limitaciones del proyecto

Las principales limitaciones identificadas en la Línea A son las siguientes:

- La muestra etiquetada contiene 1.500 comentarios, por lo que las métricas pueden variar al incorporar nuevas observaciones o nuevas particiones de datos.
- Los resultados se obtuvieron con una única división train-validation-test. Una validación cruzada repetida o un esquema de validación anidada permitiría evaluar con mayor precisión la estabilidad de las configuraciones.
- No se midió acuerdo entre etiquetadores, por lo que no se dispone de un indicador formal de consistencia de las etiquetas manuales.
- Word2Vec fue entrenado sobre el corpus disponible en el experimento y no se utilizaron embeddings preentrenados externos.
- El promedio de vectores Word2Vec no conserva explícitamente el orden de las palabras ni la estructura sintáctica de los comentarios.
- La ambigüedad, brevedad y posible superposición semántica entre categorías pueden incrementar falsos negativos y falsos positivos.

Las limitaciones específicas de la Línea B son las siguientes:

- Las etiquetas provienen de columnas binarias predefinidas en el dataset, sin etiquetado manual ni verificación de consistencia entre anotadores.
- El pool multiclase presenta un desbalance marcado: **atencion** concentra el 87 % de los casos, lo que afecta el F1 macro en el escenario semi-supervisado y puede sesgar la propagación de etiquetas en self-training.
- Se excluyeron los comentarios con cero tópicos o más de un tópico activo (18.002 de los 40.000 de la muestra), reduciendo el tamaño efectivo del experimento.
- Word2Vec fue entrenado sobre la muestra disponible y no se utilizaron embeddings preentrenados externos.
- El umbral de self-training (0.80) es fijo; un umbral adaptativo podría mejorar la propagación de etiquetas en clases minoritarias.

11. Conclusiones y trabajo futuro

El proyecto integró dos desarrollos paralelos para el análisis de comentarios abiertos. En ambas líneas se contemplan etapas de estructuración de datos, limpieza y normalización textual, generación de representaciones, entrenamiento de modelos, ajuste de hiperparámetros, evaluación y análisis de resultados. Esta organización permite abordar las particularidades de cada dominio sin perder una estructura metodológica común.

En la Línea A, el principal hallazgo fue que TF-IDF con n-gramas obtuvo mejor desempeño que Word2Vec en los diez experimentos comparados. Antes del ajuste de umbrales, el mejor resultado observado en prueba correspondió a TF-IDF sobre texto lematizado, con cinco vecinos, ponderación por distancia y métrica Minkowski con $p = 2$. Esta configuración alcanzó F1 macro de 0.8133, precisión macro de 0.9177 y Hamming loss de 0.0717.

La etapa KNN 7 mostró que la selección final depende del criterio operativo adoptado. Al calibrar umbrales por categoría con énfasis en recall, TF-IDF con stemming alcanzó recall macro de 0.8600, precisión macro de 0.7493 y F1 macro de 0.7952. En consecuencia, TF-IDF con lematización se mantiene como el mejor modelo base por F1 macro, mientras que TF-IDF con stemming se recomienda como configuración final calibrada cuando se prioriza recuperar una mayor cantidad de comentarios pertinentes.

Los resultados de la Línea A muestran que el preprocesamiento modifica el desempeño del modelo. La lematización y el stemming fueron las variantes más competitivas, mientras que la remoción de tildes mostró un efecto reducido. Además, la categoría de información y comunicación concentró la principal oportunidad de mejora antes de calibrar los umbrales.

En la Línea B, el esquema semi-supervisado mostró que TF-IDF con stemming y SVM es la combinación más robusta en el corpus hospitalario. El self-training no mejoró respecto al supervisado en la mayoría de los casos, y la diferencia se amplió en clases con pocos representantes, lo que sugiere que el umbral de confianza y el desbalance de clases son factores críticos en este escenario. El modelo final con ajuste de hiperparámetros ($C = 10$) alcanzó F1 macro de 0.959 y exactitud de 0.995 en prueba, evidenciando una alta capacidad discriminativa cuando se dispone de etiquetas suficientes para las cinco categorías.

Como extensiones futuras del proyecto completo, se propone ampliar las muestras etiquetadas, incorporar revisión o acuerdo entre etiquetadores, aplicar validación cruzada repetida, refinar la calibración de umbrales con curvas precision-recall, evaluar selección de características y contrastar los resultados con clasificadores adicionales cuando corresponda. También sería pertinente explorar embeddings contextualizados y evaluar la estabilidad de las configuraciones con nuevos datos de cada dominio. Una vez que los clasificadores base de cada línea hayan sido validados con los criterios definidos, podrían incorporarse estrategias de aprendizaje semi-supervisado con mecanismos explícitos de control de confianza [3, 4].

12. Reproducibilidad

El repositorio del proyecto debe almacenar los notebooks, las bases de etiquetado, los reportes de clasificación, las tablas de resultados y el código fuente de este informe. La trazabilidad de cada línea de trabajo se mantiene mediante la documentación de las configuraciones, variables, semillas aleatorias, particiones y métricas reportadas.

Referencias

- [1] Kowsari, K., Jafari Meimandi, K., Heidarysafa, M., Mendu, S., Barnes, L., & Brown, D. (2019). Text classification algorithms: A survey. *Information*, 10(4), 150. <https://doi.org/10.3390/info10040150>
- [2] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [3] scikit-learn developers. (s.f.). *Semi-supervised learning*. https://scikit-learn.org/stable/modules/semi_supervised.html
- [4] scikit-learn developers. (s.f.). *SelfTrainingClassifier*. https://scikit-learn.org/stable/modules/generated/sklearn.semi_supervised.SelfTrainingClassifier.html